

Quantifying the Code-Size Overhead of eBPF JIT Compilation

Hoang Duong, Hao Sun, Zhendong Su

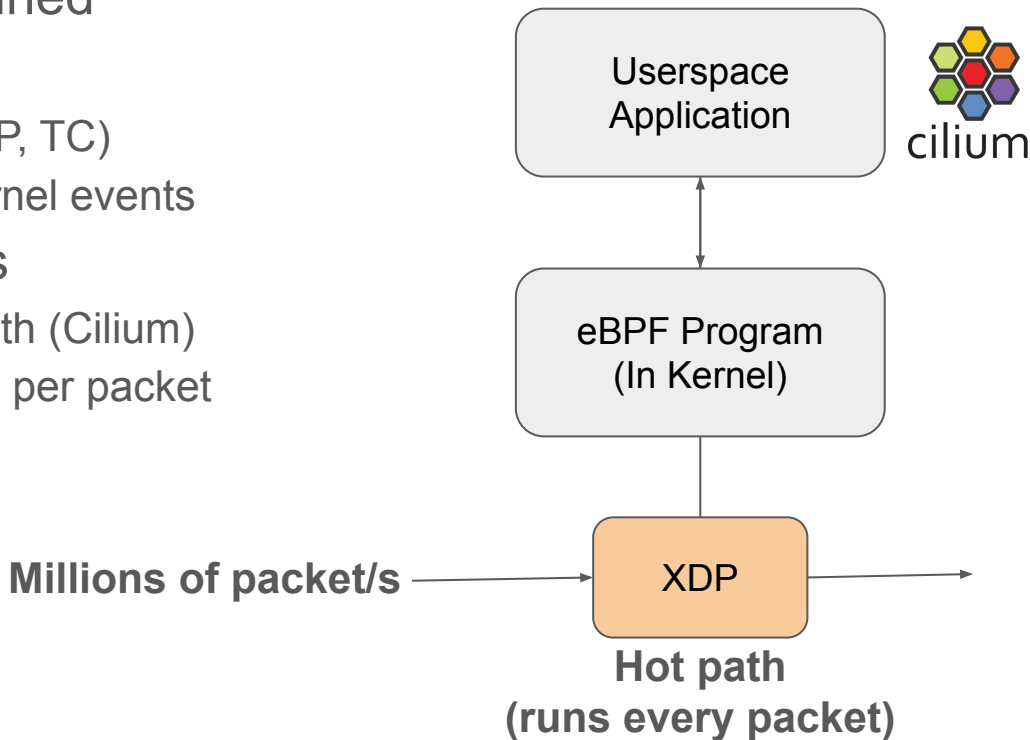


ÉCOLE
POLYTECHNIQUE

ETH zürich

eBPF in Kernel's Hot Paths

- eBPF runs verified user-defined code as kernel extensions
 - Network packet handling (XDP, TC)
 - Tracking system calls and kernel events
- Where performance matters
 - Runs directly in the packet path (Cilium)
 - Every extra instruction counts per packet



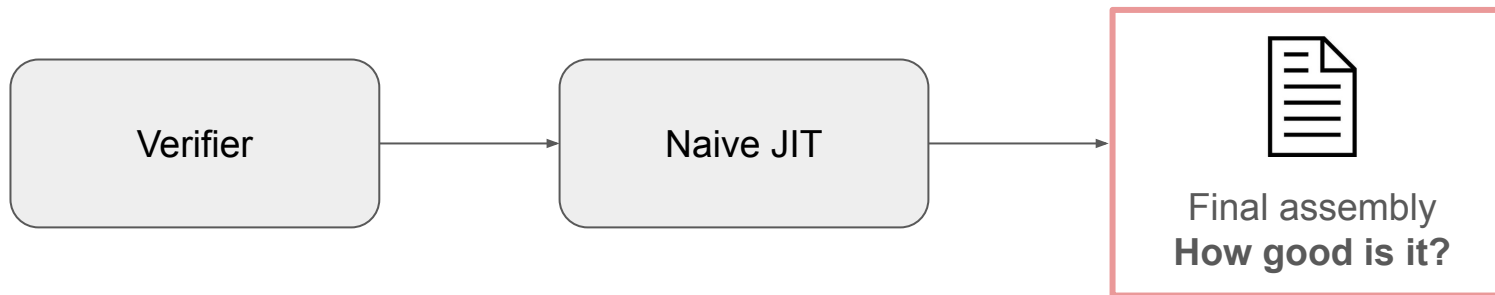
Constraints of Optimization

- **Verifier:** Ensuring safety also brings false positive, making certain compiler optimizations harder to use
- **Restricted ISA:** Only 11 registers, no sub-register operations, limited instructions for backend
- **One-to-One JIT:** Bytecode maps almost 1-to-1, no optimization
- Other side-effects in frontend



How Efficient is the Final code?

- Code quality and code size matters inside the kernel
 - Can lead to verifier rejections
- Yet, no robust way to measure and quantify
 - Prior works focused on optimizing the bytecode, not measuring the final JIT
- **Question:** How large is the overhead in real programs than necessary? At which stage does it appear?



Example: Same C code may cost more in eBPF

Example 1: Comparison of 1 byte

Normal Compiler (x86)

```
cmpb $1, (%rdi)
```

eBPF JIT Output

```
movzbl (%rdi), %eax ; widen to 32 bits  
andl $0xff, %eax ; mask back to 8 bits  
cmpl $1, %eax
```

- No 8-bit comparison → needs 3 instructions instead of 1

Example: Same C code may cost more in eBPF

Example 2: Copying an IPv6 address

Normal Compiler (x86)

```
movq (%rsi), %rax ; load
movq %rax, (%rdi) ; store
movq 8(%rsi), %rax ; load
movq %rax, 8(%rdi) ; store
```

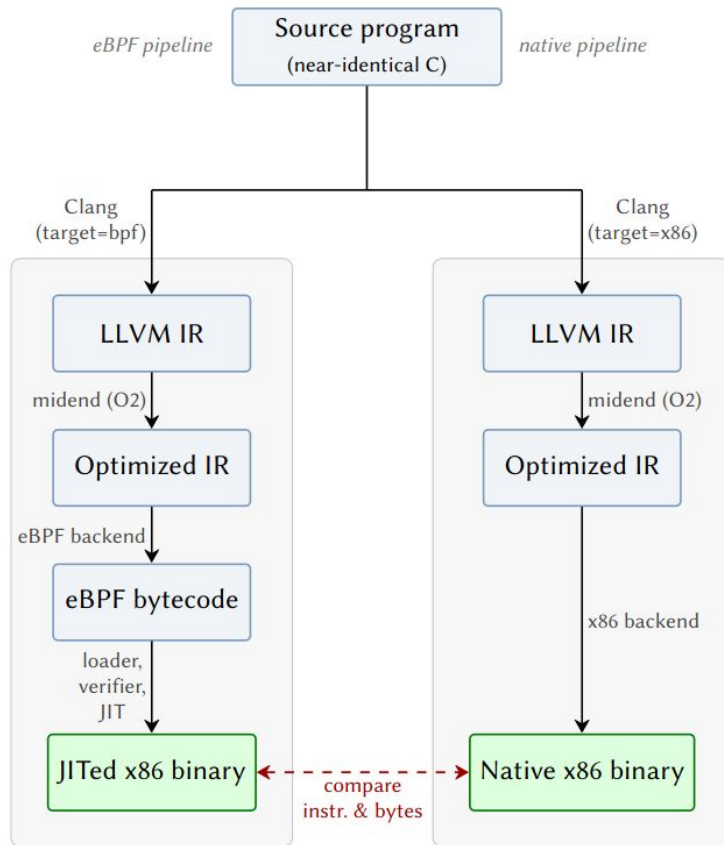
eBPF JIT Output

```
movzbb (%rsi), %rdx ; load 1 byte
shll $8, %edx ; shift it
movzbb 1(%rsi), %rax ; load next byte
orq %rax, %rdx ; combine them
... ; repeats for all 16 bytes
```

- Expands large moves into one byte at a time, over **40 instructions**
- In Cilium, these instances happen numerous times

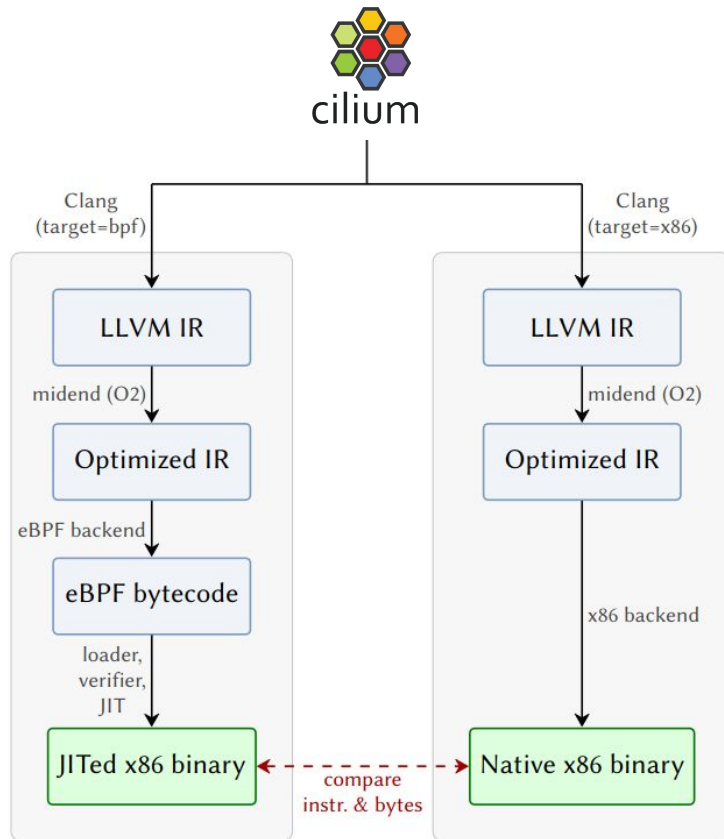
Methodology: Compare Against a Native Build

- **Native Baseline:** Same C code source, compile two ways at 3 stages
- Frontend and O2 identical, differ only in target and backend
- Need to ensure a fair native baseline



Setup: Cilium Datapath Objects

- Experimented on Cilium's 7 Datapath objects
 - Large dataset of real per-packet networking code
- Linux 6.19 (bpf-next), LLVM 21.1.8, x86-64, Cilium 1.20-dev
- **Metrics:** Instruction count and byte size at frontend, midend, backend



Ensuring Fairness in Two Divergent Pipelines

- eBPF-specific construct
- Native Target settings
- In-kernel verifier rewrites
- Dead-code Removal

Ensuring Fairness: eBPF-specific construct

- Most eBPF constructs such as eBPF maps and helpers are lowered to instructions of similar size
 - LLVM native path fills the kernel addresses with temporary relocation info
- However, Cilium specifically uses inline assembly of BPF bytecode for verifier-friendly purposes
 - Rewritten to match

```
#ifdef __bpf__ // eBPF: original eBPF asm
asm("%[i] <= 2;"
    "if %[i] > %[l] goto +1;"
    "%[a] += %[i];"
    "%[d] = *(u32*)(%[a])");
#else // native: x86, same size
asm("shlq $2,%[i];"
    "cmpq %[l],%[i]; ja 1f;"
    "addq %[i],%[a];"
    "1: movl (%[a]),%[d]");
#endif
```

Ensuring Fairness: Native Target settings

- Match kernel environments and code structure by disabling certain x86-64 features not in eBPF, e.g.
 - e.g. SIMD, vectorization, PIC, enforcing large memory model

Ensuring Fairness: In-kernel Verifier rewrites

- Verification modifies the program before JIT
 - Manually inserting checks, e.g. zero-division check, pointer boundary, etc.
 - Inline basic helper calls e.g. `map_lookup_elem`
- Solution: Patched kernel to disable auxiliary runtime rewrites, closely align more with native
 - Verifier dead-code elimination intact (see next slide)

No rewrite

```
call bpf_map_lookup_elem
```

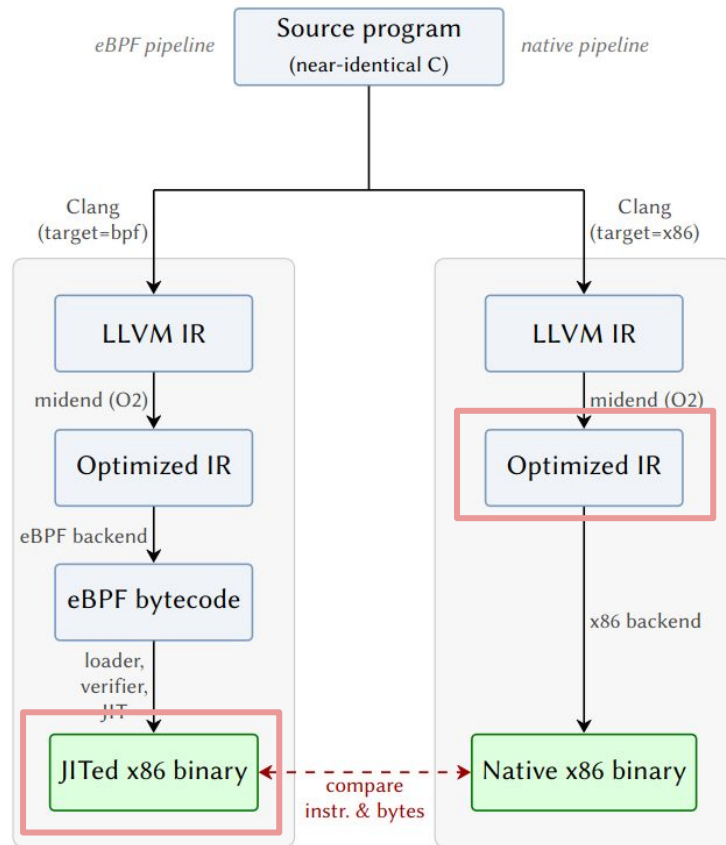


Verifier rewrites

```
movl (%rsi), %eax  
cmpq $256, %rax ; index < max_entries?  
jae miss  
shlq $3, %rax ; index * value size  
addq %rdi, %rax ; + map base
```

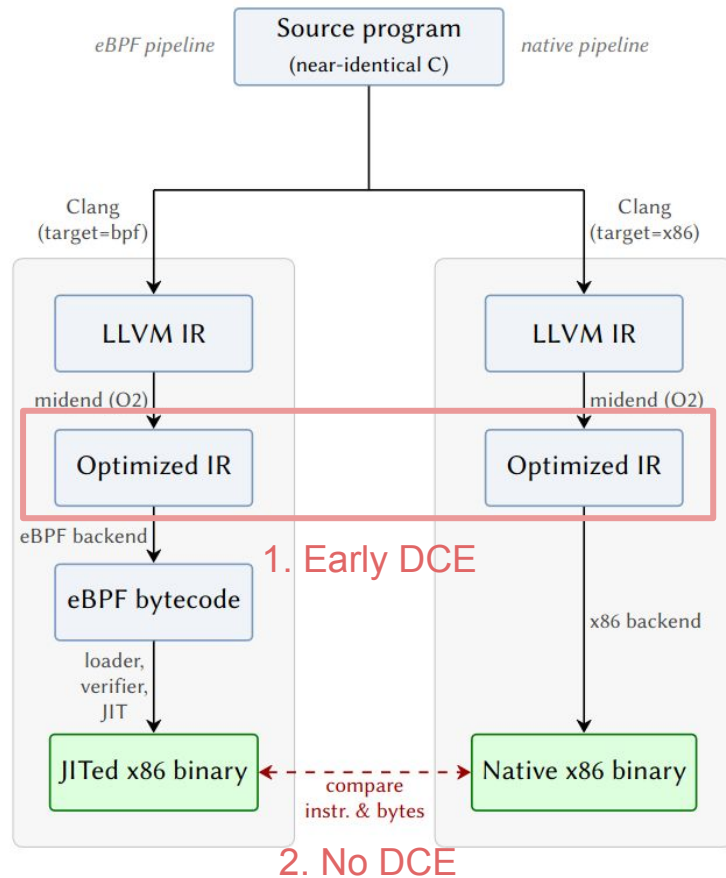
Ensuring Fairness: Dead-code Elimination (DCE)

- Cilium heavily utilizes the pattern that delays dead-code removal to the verifier instead of during compilation
 - `volatile const` + readonly section
 - Allows Cilium's configuration to be set at loadtime and DCE
- Meanwhile, native has no equivalent, only two scenarios
 - No DCE
 - Early DCE (in optimization stage)



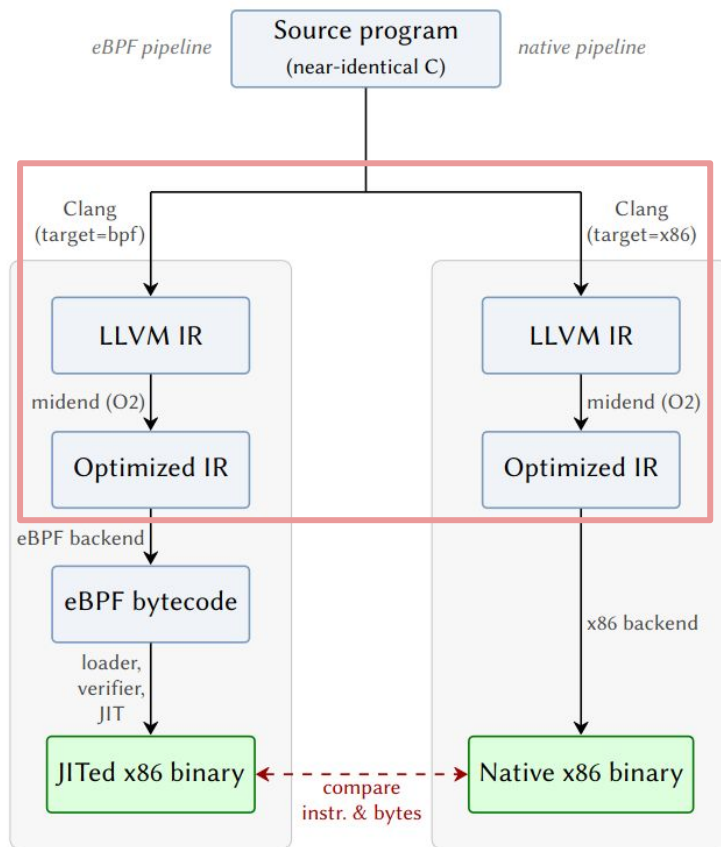
Ensuring Fairness: Dead-code Elimination (DCE)

- For fair stage-by-stage comparison, we tested both extremes
 - Also enforcing BPF pipeline to follow



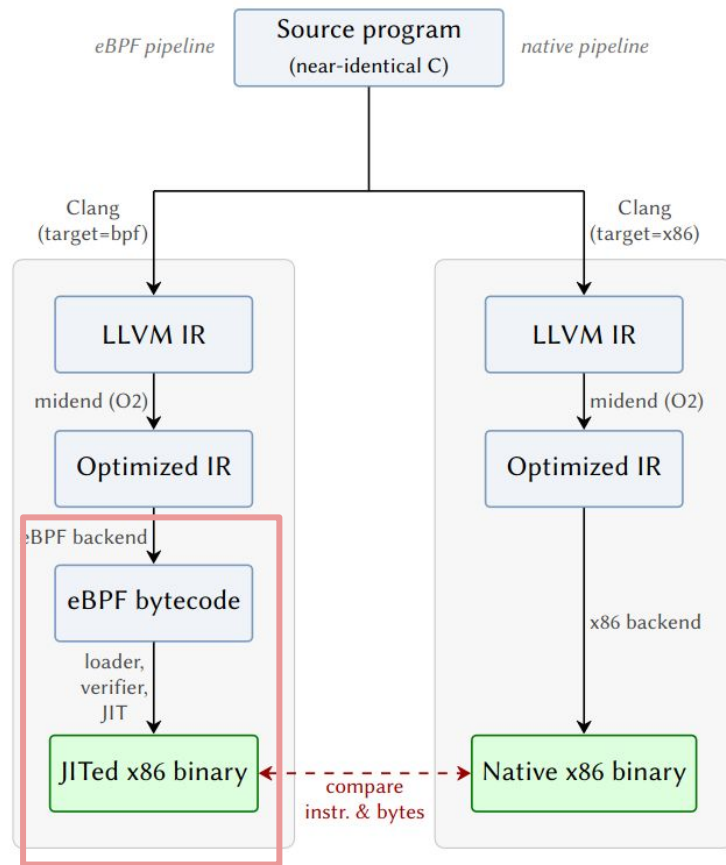
Measurement Workflow

- Frontend & Optimization midend: same process, differing in target architecture
 - Frontend: O0, no optimizations
 - Optimzation: Rerun with O2
- LLVM IR analysis and stats



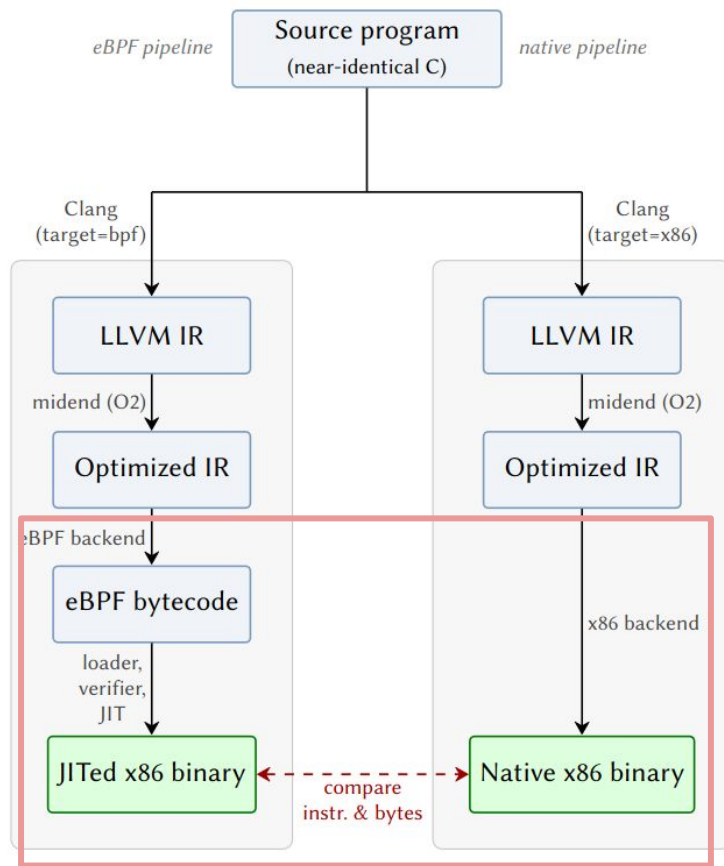
Measurement Workflow

- eBPF backend: Target BPF bytecode, track stats with `lbc`
- Load and dump JITed code with `bpftool`



Measurement Workflow

- Native backend: x86 binary and objdump
- Instruction and byte analysis with Python toolings



Result: DCE does not affect divergence

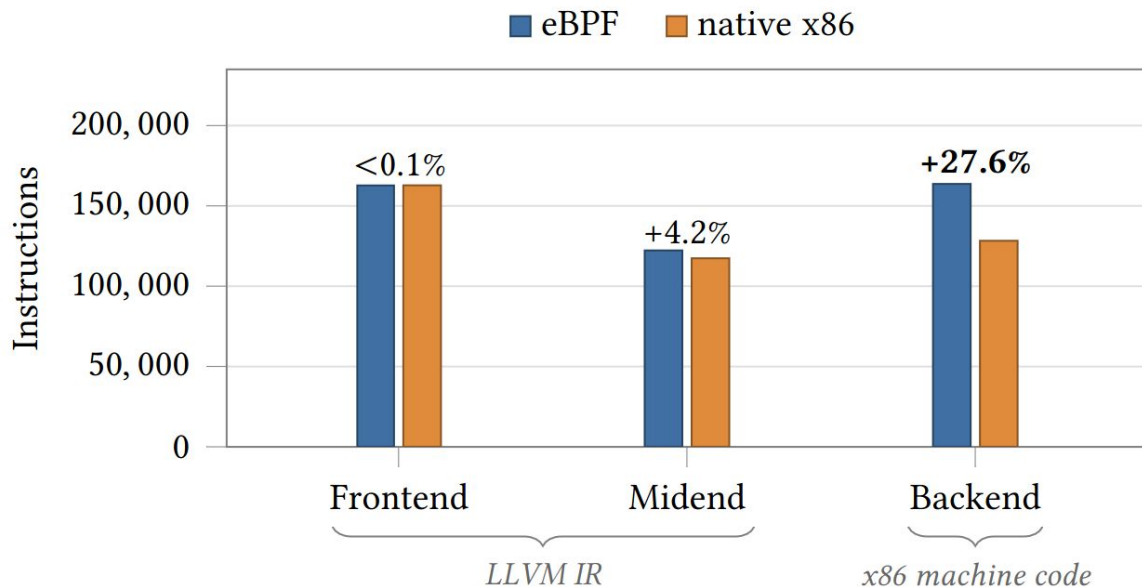
- Comparing post-O2 IR and final machine instructions across Early DCE and No DCE

Setup	Pipeline	Midend IR (-O2)	Machine Instructions	Byte Size
Early DCE (Branches pruned early)	eBPF	122,242	163,629	682,708
	Native	117,350	128,216	585,510
	Gap	+4.2%	+27.6%	+16.6%
No DCE (Dead branches kept)	eBPF	93,969	128,901	533,866
	Native	89,822	100,106	445,353
	Gap	+4.6%	+28.8%	+19.9%

→ Conclusion: Timing of DCE virtually does not affect the gap

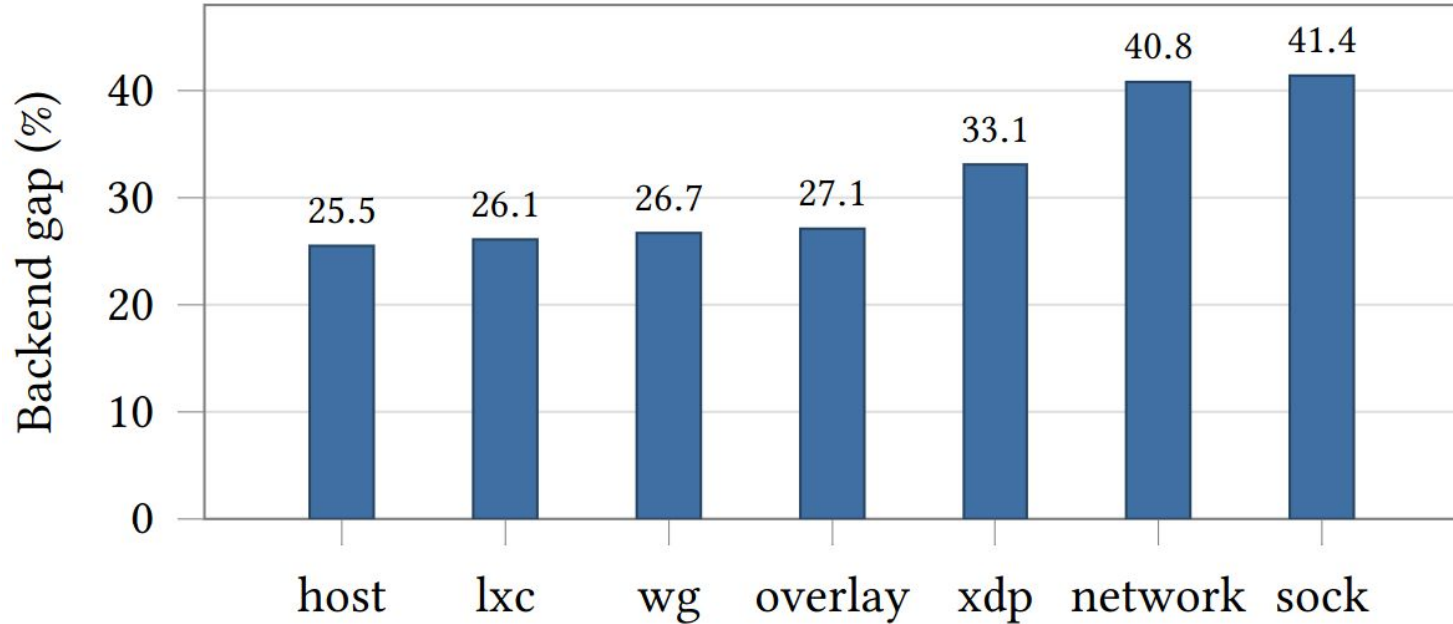
We will focus on Early DCE as it better reflects the real code

Result: Stage-by-Stage Comparison



→ Code stays fairly indistinguished until backend code generation

Result: Consistent Across All Objects



→ The effect happens on all 7 Cilium programs

Result: What Type of Instructions

Category	eBPF JIT	Native x86	Difference	% Increase
Data Movement	80,558	70,070	+10,488	+15.0%
Arithmetic	11,656	5,552	+6,104	+109.9%
Logic / Bitwise	26,315	12,942	+13,373	+103.3%
Control Flow	40,968	37,497	+3,471	+9.3%
Stack Operations	3,342	1,516	+1,826	+120.4%
Other	790	639	+151	+23.6%
Total	163,629	128,216	+35,413	+27.6%

Attributing the Gap

Based on the results, we can group each sources by the root causes:

- **ISA-Bound**
 - Hard limitation of eBPF's instruction set
- **Pipeline-structural**
 - Feature unimplemented by LLVM-backend

Attributing the Gap: ISA-Bound

- Many instructions gap attributed by the instruction analysis, resulting in expansions
 - Addressing mode (`leaq`)
 - Sub-32-bit ALU (`cmpb`, `testb`, `andb`)
 - Conditional moves (`cmov`, `setcc`)
- Resulting extra data move (`movzbq`) and logic (`andl`) instructions

Normal Compiler (x86)

```
leaq 0x20(%rdi,%rax,4), %rcx
```

eBPF JIT Output

```
movq %rdi, %rcx  
imulq $4, %rax  
addq %rax, %rcx  
addq $0x20, %rcx
```

Attributing the Gap: ISA-Bound

- Many instructions gap attributed by the instruction analysis, resulting in expansions
 - Addressing mode (`leaq`)
 - Sub-32-bit ALU (`cmpb`, `testb`, `andb`)
 - Conditional moves (`cmov`, `setcc`)
- Resulting extra data move (`movzbq`) and logic (`andl`) instructions

Normal Compiler (x86)

```
leaq 0x20(%rdi,%rax,4), %rcx
```

eBPF JIT Output

```
movq %rdi, %rcx  
imulq $4, %rax  
addq %rax, %rcx  
addq $0x20, %rcx
```

Attributing the Gap: Pipeline-structural

- Part of the gap can be attributed to missed backend optimizations
- Byte-wise struct copies: Kernel supports 8-byte operations, yet eBPF copies byte-by-byte

Normal Compiler (x86)

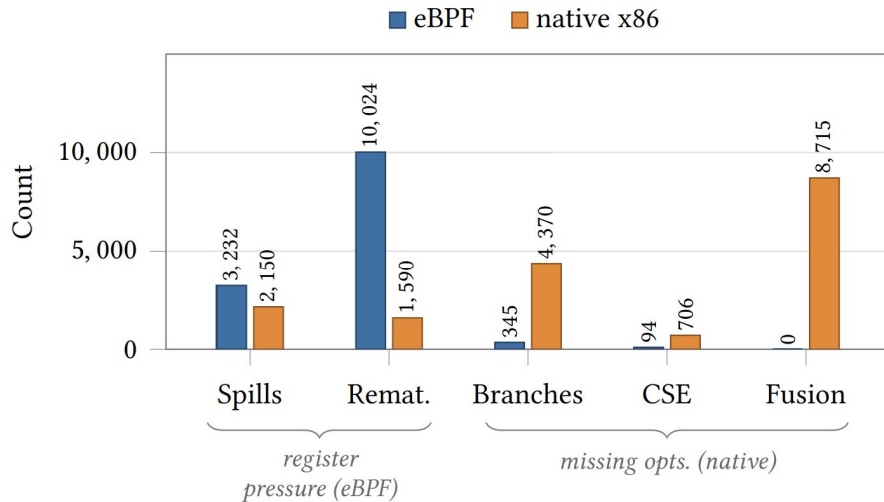
```
movq (%rsi), %rax ; load
movq %rax, (%rdi) ; store
movq 8(%rsi), %rax ; load
movq %rax, 8(%rdi) ; store
```

eBPF JIT Output

```
movzbq (%rsi), %rdx ; load 1 byte
shll    $8, %edx    ; shift it
movzbq 1(%rsi), %rax ; load next byte
orq     %rax, %rdx  ; combine them
...      ; repeats for all 16 bytes
```

Attributing the Gap: Pipeline-structural

- Register pressure: Large increase in stack operations and backend stack stats (llc)
- ISA-Bound: Less registers from eBPF ISA causing more spills
- However, rematerialization occurs more 6 times



Discussions

Each category calls for different fixes, though the patterns are not independent

- **ISA Fixes:**
 - Supporting more instructions and registers in the kernel in future eBPF versions
- **Compiler fixes:**
 - Port more opt passes to eBPF backend (might be verifier-limited)
 - Detect wide byte moves
 - Better register management

Discussions: Runtime cost

- Code size is only a partial proxy for runtime
 - Instructions costing more micro-ops
 - Scheduling, macro-ops fusion
 - However, measuring runtime gap is not viable
 - Native baseline is not a runnable or loadable program
 - Would need a custom kernel or loader to execute in the kernel
- Limitation of the framework

Conclusion

- We built a fair baseline to measure eBPF code size against native LLVM output, showing overhead mostly appears in backend
- Part of the gap is responsible by ISA limitations, but a large part is also pipeline-structural, which can be fixed in LLVM today
- The framework also contributes a method to identify overhead and areas to improve

Conclusion

- Future works:
 - Runtime comparison
 - Workloads other than Cilium
 - Other target architectures (Arm64, Risc-V)
 - Other compiler backend (GCC)